

Automatización en la Asignación de Tareas en un ERP para Talleres Industriales utilizando Algoritmos Genéticos



Pontificia Universidad
JAVERIANA
Cali

[VIGILADA MINEDUCACIÓN Res. 12230 de 2016.]

Maria José Pava Echeverry

Jose Daniel Ramirez Delgado

Directora:

PhD. Gloria Ines Alvares Vargas

Co-director:

PhD. Diego Luis Linares Ospina

FACULTAD DE INGENIERÍA Y CIENCIAS
INGENIERÍA DE SISTEMAS Y COMPUTACIÓN

Pontificia Universidad Javeriana Cali

Santiago de Cali, 2025

Índice general

Índice de figuras

Índice de tablas

Capítulo 1

Introducción

El presente proyecto de grado se centra en abordar los desafíos que enfrentan los talleres industriales en la gestión eficiente de sus operaciones, específicamente en la asignación de tareas dentro de un sistema de Planificación de Recursos Empresariales (ERP). La asignación tradicional de tareas, frecuentemente realizada de forma manual, puede conducir a ineficiencias significativas, tales como la sobrecarga de operarios, tiempos muertos en la producción y retrasos en las entregas, afectando directamente la productividad y la rentabilidad del taller.

En este contexto, la automatización de la asignación de tareas emerge como una solución crucial para optimizar el uso de los recursos y mejorar el flujo de trabajo. Este proyecto propone el desarrollo de un modelo de automatización basado en algoritmos genéticos, una técnica de inteligencia artificial que ha demostrado su eficacia en la resolución de problemas complejos de optimización. Al implementar algoritmos genéticos en el módulo de gestión de órdenes de trabajo de un ERP, se busca automatizar la distribución de tareas considerando variables clave como las capacidades técnicas de los operarios, la disponibilidad de insumos y la carga de trabajo actual.

Además de desarrollar este modelo de automatización, el proyecto tiene como objetivo evaluar su desempeño en comparación con otras técnicas de automatización utilizadas en el campo de la ingeniería industrial. Esta comparación permitirá validar la eficacia y aplicabilidad del enfoque propuesto en un entorno industrial real, asegurando la selección de la estrategia más adecuada para la optimización de la asignación de tareas.

La investigación se llevará a cabo en colaboración con la empresa Rectificadora Recti-Ram, lo que proporcionará un contexto real y datos relevantes

para el desarrollo y la validación del modelo.

A través de este proyecto, se espera contribuir al avance en la automatización de los talleres industriales, proporcionando una herramienta que permita mejorar la eficiencia operativa, reducir los costos y aumentar la satisfacción del cliente mediante la optimización de los procesos de asignación de tareas.

Capítulo 2

Definición y Planteamiento del Problema

2.1. Planteamiento del Problema

El entorno productivo de los talleres industriales enfrenta diversas problemáticas que afectan su eficiencia y rendimiento. Entre los principales desafíos se encuentran la capacidad de producción limitada, los largos tiempos de fabricación y los retrasos en las fechas de entrega [1]. Además, la falta de sistemas estructurados para la evaluación de calidad y la baja integración tecnológica pueden impactar negativamente la productividad del proceso [1].

La asignación de tareas en los talleres industriales es un factor clave para optimizar el uso de recursos humanos y técnicos. Una planificación ineficiente puede provocar retrasos, una distribución desequilibrada de la carga de trabajo y una reducción en la productividad [2]. La complejidad de estos entornos productivos, caracterizados por la variabilidad en la demanda y la necesidad de coordinar múltiples operaciones, hace necesario el uso de enfoques avanzados que permitan gestionar eficazmente la distribución de actividades y mejorar el cumplimiento de los plazos de entrega [2].

Para abordar este problema, se propone la implementación de algoritmos genéticos (AG) en un módulo ERP de gestión de órdenes de trabajo. Esta solución permitirá automatizar la distribución de tareas considerando factores clave como:

- Capacidades técnicas de cada operario.

CAPÍTULO 2. DEFINICIÓN Y PLANTEAMIENTO DEL PROBLEMA 4

- Disponibilidad de insumos requeridos para cada tarea.
- Carga de trabajo actual y disponibilidad horaria de los operarios.

Adicionalmente, este estudio evaluará el desempeño del algoritmo genético en comparación con otro método de optimización utilizado en ingeniería industrial, con el objetivo de determinar su viabilidad y eficacia en este contexto.

2.1.1. Formulación

¿Cómo mejorar la asignación de tareas en un ERP de talleres industriales utilizando algoritmos genéticos, y cómo se compara su rendimiento con otras técnicas de automatización utilizadas en ingeniería industrial?

2.1.2. Sistematización

Para abordar de manera integral el problema de investigación, se plantean las siguientes preguntas:

1. ¿Cuáles son los principales factores que afectan la asignación de tareas en talleres industriales?
2. ¿Cómo pueden los algoritmos genéticos mejorar la automatización de tareas dentro del ERP?
3. ¿Qué otras técnicas de automatización existen en ingeniería industrial para resolver problemas similares?
4. ¿Cómo se comparan los resultados obtenidos con algoritmos genéticos respecto a otra técnica de automatización en términos de eficiencia y calidad de la solución?

2.2. Objetivos

2.2.1. Objetivo General

Desarrollar un modelo de automatización basado en algoritmos genéticos para apoyar en el módulo de asignación de tareas de un sistema ERP de talleres industriales, comparándolo con otro método de automatización utilizado en ingeniería industrial.

2.2.2. Objetivos Específicos

Para alcanzar el objetivo general, se plantean los siguientes objetivos específicos:

1. Identificar las características claves que influyen en la asignación de tareas en talleres industriales.
2. Diseñar e implementar un modelo utilizando un algoritmo genético para automatizar la asignación de tareas tales como desmontaje del motor, limpieza y diagnóstico, rectificación de cilindros, rectificación de cigüeñales, ajuste de válvulas, ensamblaje y prueba del motor.
3. Evaluar la funcionalidad del modelo y el desempeño del algoritmo genético en comparación con otras técnicas de automatización utilizadas en ingeniería industrial.
4. Analizar los beneficios y limitaciones de los algoritmos genéticos en este contexto.

2.3. Justificación

La gestión eficiente de órdenes de trabajo en talleres industriales impacta directamente la productividad y rentabilidad. El uso de algoritmos genéticos puede ofrecer una solución flexible y adaptable que automatiza los tiempos de ejecución, minimiza costos y mejora la distribución equitativa de la carga de trabajo.

La empresa Rectificadora Recti-Ram cuenta con datos históricos sobre tiempos de reparación, asignación de operarios y disponibilidad de recursos, lo que facilita el entrenamiento y validación del modelo propuesto. Asimismo, la disponibilidad de la empresa para proporcionar información clave sobre su flujo de trabajo y sus restricciones operativas permitirá diseñar un modelo que se ajuste a las necesidades reales.

Además, comparar esta solución con otro método de automatización permitirá validar su aplicabilidad en escenarios industriales reales, asegurando que la mejor estrategia sea implementada dentro del ERP del taller, garantizando así una planificación eficiente y adaptable a las condiciones dinámicas del entorno productivo.

2.4. Delimitaciones y Alcances

- Se enfocará en la asignación de tareas dentro del módulo de ordenes de trabajo en un ERP para talleres industriales.
- Se implementará un modelo basado en algoritmos genéticos.
- Se comparará con otro método de automatización (a definir en consulta con expertos en ingeniería industrial).

Capítulo 3

Marco de Referencia

3.1. Marco Teórico

Para comprender el desarrollo de un módulo de ERP destinado a la automatización de la asignación de tareas en un taller industrial, es esencial establecer una base teórica sólida sobre los conceptos clave involucrados. Este proyecto se enfoca en la aplicación de algoritmos genéticos como una técnica de inteligencia artificial que permite la automatización de procesos de toma de decisiones en entornos industriales. Los algoritmos genéticos permiten analizar múltiples combinaciones posibles para la asignación de recursos y actividades, facilitando la gestión eficiente de órdenes de trabajo en un ERP.

Dentro del campo de la ingeniería industrial, se han desarrollado diversas metodologías para mejorar la productividad y la gestión de recursos, muchas de ellas basadas en principios de automatización y sistemas inteligentes. En este proyecto, se desarrollará un módulo de ERP que empleará algoritmos genéticos para la automatización de la asignación de tareas, permitiendo reducir la intervención manual y mejorar la eficiencia operativa en talleres industriales. A lo largo del estudio, se comparará este enfoque con otras metodologías existentes en la industria, evaluando su desempeño en términos de tiempos de procesamiento, flexibilidad y adaptabilidad a escenarios reales.

Los sistemas ERP (Enterprise Resource Planning) son herramientas integradas que permiten la automatización y gestión de los procesos empresariales esenciales, tales como finanzas, producción, logística y ventas. Un ERP centraliza la información empresarial, optimizando la toma de decisiones y reduciendo la fragmentación de datos, lo que mejora la eficiencia operativa. Estos sistemas

han evolucionado desde los sistemas MRP (Material Requirements Planning) y actualmente constituyen una pieza clave en la digitalización de la industria [3].

3.1.1. Automatización con ERP en talleres industriales

La implementación de ERP en talleres industriales responde a la necesidad de automatizar y estructurar procesos que, de otro modo, se gestionarían de forma manual, lo que puede generar ineficiencias. En los talleres de tecno-mecánica, donde se requiere un control riguroso de los repuestos y tiempos de trabajo, un ERP automatizado permite la asignación eficiente de tareas según la disponibilidad de operarios y recursos, reduciendo tiempos de espera y mejorando la productividad.

En un estudio realizado en 2019 [4], se identificó que el 70.4 % de las quejas de los clientes en una compañía analizada estaban relacionadas con retrasos en la entrega de órdenes de trabajo, mientras que el 29.6 % restante correspondía a problemas de órdenes no confirmadas y reprocesos. Estos problemas, comunes en talleres industriales, pueden ser mitigados mediante un ERP que automatice la distribución de tareas y optimice la administración de recursos, minimizando errores humanos y mejorando la trazabilidad de las órdenes de trabajo.

3.1.2. Algoritmos Genéticos (GA) para la Automatización

La automatización de procesos en entornos industriales ha avanzado con el uso de algoritmos evolutivos, entre los cuales los algoritmos genéticos (GA) han demostrado ser eficaces para resolver problemas de asignación de tareas y planificación de recursos. Estos algoritmos, inspirados en la selección natural y la evolución biológica, trabajan con poblaciones de soluciones potenciales, aplicando operadores como la selección, el cruce (crossover) y la mutación para encontrar soluciones óptimas de manera iterativa [5].

A diferencia de los métodos tradicionales de asignación de tareas, que pueden depender de reglas predefinidas y cálculos deterministas, los algoritmos genéticos ofrecen una solución flexible que se adapta dinámicamente a cambios en el entorno de producción. Gracias a su capacidad de explorar múltiples combinaciones de asignación de tareas y su enfoque probabilístico, los GA

pueden reducir tiempos improductivos y mejorar la distribución del trabajo en talleres industriales.

3.2. Técnicas de Automatización en Ingeniería Industrial

La automatización en ingeniería industrial comprende un conjunto de metodologías, técnicas y herramientas orientadas a optimizar el uso de recursos, reducir desperdicios y mejorar el rendimiento operativo en entornos productivos. Diversos estudios han señalado que la mejora de la eficiencia en talleres y plantas manufactureras depende directamente de la capacidad para programar tareas, asignar recursos y minimizar tiempos improductivos [6, 7]. En los talleres industriales modernos, donde existe alta variabilidad de tareas y heterogeneidad en los tiempos de ejecución, la automatización se convierte en un elemento clave para garantizar un flujo de trabajo continuo y una gestión eficiente de las órdenes de trabajo [8].

Estas técnicas abarcan desde enfoques de mejora continua, como Lean Manufacturing y Six Sigma, hasta métodos formales de secuenciación basados en reglas de prioridad —tales como SPT, FCFS y EDD— que constituyen la base histórica de la programación industrial [9, 10]. Asimismo, la literatura reciente destaca la importancia de combinar estas metodologías con algoritmos avanzados de optimización, dada la complejidad creciente de los sistemas productivos y la naturaleza NP-hard de muchos problemas de scheduling [6, 8].

A continuación se presentan las principales técnicas de automatización utilizadas en la industria para la asignación y programación de tareas, todas ellas relevantes como puntos de comparación frente a métodos evolutivos como los algoritmos genéticos.

3.2.1. Lean Manufacturing y mejora continua

Lean Manufacturing es una filosofía de gestión orientada a la eliminación sistemática de desperdicios (*muda*) y a la optimización del flujo de trabajo mediante la mejora continua. Sus principios, derivados del Sistema de Producción Toyota, se aplican en talleres industriales para reducir tiempos improductivos, minimizar inventarios y mejorar la estabilidad del proceso productivo. La literatura en secuenciación de operaciones y en sistemas de

manufactura destaca que los talleres que adoptan prácticas Lean presentan niveles más estables de procesamiento y menor variabilidad temporal en sus operaciones [7, 6].

En entornos de tecnomecánica y mantenimiento industrial, Lean permite estructurar procesos que tradicionalmente se ejecutan de forma manual o desorganizada, facilitando la estandarización y priorización de tareas. Herramientas como 5S, Value Stream Mapping y Kanban han demostrado ser efectivas para mejorar la trazabilidad de órdenes de trabajo, reducir cuellos de botella y aumentar la utilización de operarios y máquinas [11]. Estas prácticas aportan la base conceptual necesaria para implementar métodos de automatización más avanzados dentro de un ERP, permitiendo que reglas de despacho y algoritmos de optimización operen sobre flujos de trabajo más consistentes.

3.2.2. Six Sigma y reducción de variabilidad

Six Sigma es una metodología orientada a reducir la variabilidad de los procesos mediante técnicas estadísticas avanzadas y un enfoque basado en datos. Su ciclo DMAIC permite analizar de manera estructurada las causas de ineficiencias y establecer mejoras que impactan directamente la estabilidad del sistema productivo. La investigación en sistemas de manufactura reporta que Six Sigma mejora la precisión en los tiempos de proceso y disminuye la ocurrencia de operaciones re-trabajadas o reprocesos [6].

En talleres de mantenimiento automotriz o metalmecánica, Six Sigma facilita la estandarización de tiempos de reparación, la identificación de cuellos de botella y la reducción de variaciones entre operarios, contribuyendo a un entorno más predecible para la programación de tareas. La reducción de variabilidad es fundamental para que los métodos de scheduling —tanto heurísticos como metaheurísticos— puedan operar bajo condiciones de mayor estabilidad y precisión [8, 9].

3.2.3. Investigación de Operaciones aplicada al Scheduling

La Investigación de Operaciones (IO) ha desarrollado modelos matemáticos para resolver problemas de asignación, secuenciación y optimización de recursos. En el ámbito de la producción industrial, estos modelos incluyen:

- Programación Lineal Entera Mixta (MILP)
- Branch and Bound
- Programación Dinámica
- Modelos deterministas de Flow Shop, Job Shop y Flexible Job Shop

Estos problemas son ampliamente conocidos por su complejidad computacional, siendo Job Shop y Flexible Job Shop NP-hard [6, 7]. Si bien los modelos exactos permiten obtener soluciones óptimas, en la práctica su uso se limita a problemas de pequeña escala debido al crecimiento exponencial del espacio de búsqueda. Investigaciones recientes han demostrado que, incluso con técnicas avanzadas como Constraint Programming, resolver problemas de scheduling con múltiples restricciones sigue siendo un reto significativo para sistemas reales [12].

Por esta razón, en escenarios industriales complejos se emplean heurísticas, reglas de despacho y metaheurísticas que permiten obtener soluciones de buena calidad en tiempos computacionales reducidos [9, 10].

3.2.4. Reglas de prioridad (Dispatching Rules)

Las reglas de prioridad constituyen uno de los métodos clásicos más utilizados para la programación de tareas en entornos industriales debido a su simplicidad, bajo coste computacional y fácil implementación. Estas reglas se basan en atributos específicos de las tareas, como tiempo de procesamiento, fecha de entrega o urgencia.

Las reglas más representativas incluyen:

- **SPT (Shortest Processing Time)**

Prioriza las tareas más cortas, reduciendo el tiempo promedio de flujo y el tiempo total en el sistema. Es una técnica particularmente eficiente en sistemas sin restricciones complejas [6]. Esta regla se utiliza como base comparativa en esta investigación.

- **FCFS (First Come, First Served)** Procesa tareas en el orden en que llegan. Aunque sencilla, puede generar tiempos improductivos o cargas desbalanceadas [7].

- **EDD (Earliest Due Date)** Minimiza el retraso (tardiness) priorizando tareas con fechas de entrega más próximas. Es una regla ampliamente usada en la industria gráfica y manufactura ligera [9].
- **LPT (Longest Processing Time)** Favorece las tareas más largas y es útil en escenarios donde se busca balancear carga entre máquinas paralelas.
- **Reglas híbridas** Combinan múltiples criterios como tiempo, urgencia y vencimiento. Son más flexibles y representan la transición entre heurísticas clásicas y metaheurísticas [10].

Estas reglas son consideradas baseline industrial y sirven como punto de referencia para evaluar técnicas más avanzadas como algoritmos genéticos o algoritmos híbridos [9, 10].

3.2.5. Métodos heurísticos clásicos

Las heurísticas permiten obtener soluciones aproximadas para problemas de scheduling de manera rápida. Aunque no garantizan optimalidad, son valiosas en entornos industriales por su eficiencia computacional.

Entre las más destacadas se encuentran:

- Heurísticas de cuello de botella
- Algoritmos constructivos como NEH (Flow Shop) o G&T (Job Shop)
- List Scheduling
- Heurísticas parametrizadas por prioridad

Estudios en Flow Shop y Flexible Job Shop señalan que estas heurísticas ofrecen buen desempeño cuando el entorno productivo es estable y presenta pocas restricciones [6, 7]. Sin embargo, su desempeño disminuye considerablemente cuando existen múltiples dependencias, precedencias o recursos limitados—como es el caso en talleres industriales reales—, por lo que requieren ser complementadas con metaheurísticas [9].

3.2.6. Metaheurísticas aplicadas al scheduling

Debido a la complejidad de los problemas industriales de programación de tareas, especialmente aquellos con múltiples máquinas, precedencias y restricciones, las metaheurísticas se han convertido en herramientas fundamentales para obtener soluciones de alta calidad.

Las técnicas más utilizadas incluyen:

- Recocido Simulado (SA)
- Búsqueda Tabú (TS)
- Algoritmos Genéticos (GA)
- PSO (Particle Swarm Optimization)
- Algoritmos híbridos (GA + SA, GA + TS)

La literatura presenta numerosos casos donde los GA y los métodos híbridos superan significativamente a las heurísticas tradicionales y reglas de prioridad [9, 6]. Del mismo modo, revisiones recientes sobre programación en Industria 4.0 y 5.0 destacan el papel de las metaheurísticas en sistemas productivos dinámicos y altamente automatizados [8].

3.2.7. Métodos deterministas y exactos

Aunque su aplicabilidad es limitada en escenarios complejos, los métodos exactos cumplen un rol fundamental como referencia teórica para evaluar heurísticas y metaheurísticas.

Estos métodos incluyen:

- Branch & Bound
- Branch & Cut
- Constraint Programming (CP)
- Programación Lineal y Entera

Investigaciones recientes en CP demuestran que si bien estas técnicas logran acotar soluciones y obtener resultados cercanos al óptimo en algunos casos, su escalabilidad aún es limitada en problemas como FJSSP y JSSP [12]. Por ello, suelen emplearse solo para casos pequeños o como benchmark teórico.

3.3. Antecedentes

La programación de tareas en entornos industriales ha sido un campo de estudio ampliamente abordado en la literatura debido a la complejidad inherente de problemas como Flow Shop, Job Shop y Flexible Job Shop, todos ellos catalogados como NP-hard [6, 7]. Esta complejidad ha motivado el desarrollo de metodologías basadas en heurísticas, reglas de despacho y metaheurísticas avanzadas para obtener soluciones eficientes dentro de límites computacionales razonables. En este contexto, los algoritmos genéticos (AG) han demostrado un rendimiento destacado en la optimización de tareas, especialmente en escenarios donde múltiples restricciones interactúan entre sí.

Uno de los trabajos más relevantes es el de Delgado et al. [13], quienes propusieron un algoritmo genético para la programación de tareas en una celda de manufactura y compararon su desempeño con métodos heurísticos tradicionales. Sus resultados mostraron que los AG pueden reducir tiempos improductivos y mejorar la utilización de recursos, destacando su capacidad para explorar soluciones que las reglas de prioridad no alcanzan debido a su naturaleza determinista.

De manera complementaria, Meisel y Prado [9] desarrollaron un algoritmo genético híbrido que incorpora enfriamiento simulado, demostrando que las metaheurísticas híbridas pueden ofrecer soluciones más estables y de mayor calidad en problemas de tipo Job Shop. Este estudio respalda la idea de que los AG son especialmente adecuados para problemas con múltiples restricciones, como precedencias, máquinas en paralelo y tiempos variables de procesamiento.

En el ámbito de la automatización industrial desde la perspectiva de Industria 4.0, Zhang et al. [8] presentan un análisis del estado del arte de algoritmos evolutivos aplicados a la programación en sistemas inteligentes. Su investigación concluye que los AG, junto con técnicas híbridas de optimización, son herramientas clave para la toma de decisiones en entornos altamente dinámicos y automatizados. Esta tendencia refuerza la pertinencia del uso de AG como enfoque moderno para la gestión de recursos en talleres industriales.

En cuanto a la asignación de tareas en problemas de tipo Task Assignment Problem (TAP), Savić et al. [14] demostraron que los AG pueden encontrar soluciones óptimas o cercanas al óptimo en espacios de búsqueda complejos, superando métodos deterministas y heurísticos en términos de tiempo de ejecución y calidad de resultados. De forma similar, Wang [15] aplicó algoritmos genéticos al diseño automático de maquinaria, mostrando mejo-

ras significativas frente a métodos tradicionales en precisión y tiempos de procesamiento.

Por otra parte, investigaciones sobre heurísticas industriales, como las reglas de despacho SPT, EDD y FCFS, han demostrado ser métodos simples y eficientes para problemas básicos de secuenciación, pero con limitaciones marcadas en problemas con múltiples restricciones o estructuras complejas de precedencia [10, 6]. Esta brecha entre simplicidad y capacidad de adaptación es precisamente lo que motiva la comparación entre SPT y algoritmos evolutivos en el presente estudio.

En conjunto, estos antecedentes evidencian que los algoritmos genéticos han sido utilizados con éxito en diversos dominios de la optimización industrial, mientras que las heurísticas tradicionales siguen siendo herramientas fundamentales como métodos comparativos o de referencia. En el caso particular de un taller industrial basado en un sistema ERP, la literatura existente es limitada. Por ello, el presente proyecto aporta un enfoque novedoso al integrar un algoritmo genético dentro de un entorno simulado que replica el flujo real de un taller automotriz, comparándolo directamente con una heurística industrialmente aceptada como SPT. Esta comparación permite evaluar el potencial de los AG para mejorar la asignación de tareas y la eficiencia operativa en un contexto aplicado.

Capítulo 4

Formalización del problema

El presente capítulo describe la estructura formal del problema de programación diaria del taller industrial, incluyendo la caracterización del flujo operativo, la definición matemática de los elementos que componen el sistema y la modelación del comportamiento del taller bajo sus restricciones reales. Esta formalización constituye la base para el desarrollo posterior del algoritmo genético y el método SPT, presentados en los capítulos siguientes.

4.1. Estudio del flujo de trabajo del taller industrial

El taller de reparación de motores opera bajo un flujo secuencial condicionado por restricciones de precedencia, disponibilidad de repuestos, habilidades del personal e interacción entre tareas dentro de una misma orden de trabajo. Cada día se recibe un conjunto de órdenes de trabajo que agrupan tareas mecánicas con distintos requerimientos técnicos, y el objetivo operativo es completar la mayor cantidad posible de dichas tareas dentro de la jornada laboral.

4.1.1. Tipos de tareas

Sea $T = \{1, \dots, q\}$ el conjunto de tipos de tareas que el taller está en capacidad de ejecutar. Cada tipo de tarea representa una operación específica dentro del proceso de reparación, cuya duración base está dada por una función:

$$p : T \rightarrow \mathbb{N}$$

que asigna a cada tipo $t \in T$ su duración en minutos.
Las tareas concretas del día se representan mediante:

$$T_d = \{1, \dots, n\},$$

donde cada tarea $i \in T_d$ posee los atributos:

- $ot(i)$: orden de trabajo a la que pertenece;
- $tipo(i)$: tipo de tarea según T ;
- $p(tipo(i))$: duración efectiva.

4.1.2. Operarios y recursos disponibles

Sea $O = \{1, \dots, r\}$ el conjunto de operarios disponibles en el taller. Cada tarea requiere habilidades específicas, modeladas mediante:

$$E(t) \subseteq O, \quad \forall t \in T,$$

donde $E(t)$ es el conjunto de operarios aptos para ejecutar tareas del tipo t . La aptitud del operario es una restricción estricta: una tarea no puede ser programada por un operario fuera de $E(tipo(i))$.

4.1.3. Reglas industriales de asignación

El taller opera bajo restricciones naturales del proceso mecánico:

- **Precedencias intrínsecas:** ciertas tareas deben completarse antes de que otras puedan iniciar. Modelado mediante:

$$P(t) \subseteq T,$$

que representa los tipos de tarea que deben preceder al tipo t .

- **Precedencias por orden de trabajo:** sólo son relevantes aquellas precedencias que ocurren dentro de la misma orden de trabajo.

- **Disponibilidad de repuestos:** cada orden de trabajo ot tiene asociado un vector binario:

$$R_{ot} = (r_1, r_2, \dots), \quad r_j \in \{0, 1\},$$

donde $r_j = 1$ indica que el repuesto necesario para cierto tipo de tarea está disponible.

- **Capacidad temporal:** cada operario posee un máximo de 8 horas de trabajo:

$$H = 480 \text{ minutos.}$$

4.1.4. Análisis del proceso actual y puntos críticos

El proceso presenta dificultades estructurales:

- Los operarios poseen habilidades heterogéneas.
- Las órdenes contienen dependencias internas que condicionan el flujo.
- Los repuestos no siempre están disponibles.
- El tiempo límite de jornada restringe la cantidad de tareas ejecutables.
- Tareas pertenecientes a una misma orden no pueden solaparse.

El resultado es un entorno altamente restrictivo donde la programación manual es compleja y propensa a tiempos improductivos.

4.1.5. Flujo general del proceso

Operativamente, el taller funciona bajo este flujo conceptual:

1. Llegan varias órdenes de trabajo, cada una con un conjunto de tareas.
2. Cada tarea requiere un operario apto y, en algunos casos, un repuesto.
3. Algunas tareas dependen de otras dentro de la misma orden.
4. Los operarios ejecutan las tareas secuencialmente en una jornada limitada.
5. El objetivo es completar la mayor cantidad posible de tareas.

Este flujo será simulado formalmente en la evaluación de soluciones.

4.2. Diseño del modelo de datos para la simulación

4.2.1. Conjuntos y parámetros del sistema

El problema se formaliza mediante los siguientes conjuntos:

$$\begin{aligned} OT &= \{1, \dots, m\} && \text{Órdenes de trabajo,} \\ T_d &= \{1, \dots, n\} && \text{Tareas del día,} \\ O &= \{1, \dots, r\} && \text{Operarios,} \\ T &= \{1, \dots, q\} && \text{Tipos de tarea.} \end{aligned}$$

Parámetros principales:

$$\begin{aligned} p(t) &&& \text{Duración del tipo de tarea } t, \\ E(t) \subseteq O &&& \text{Operarios aptos,} \\ P(t) \subseteq T &&& \text{Precedencias por tipo,} \\ R_{ot} &&& \text{Vector de disponibilidad de repuestos para la OT,} \\ H = 480 &&& \text{Horizonte máximo diario.} \end{aligned}$$

4.2.2. Variables del modelo

Durante la simulación se manejan variables que representan el estado del taller:

$$\begin{aligned} tpo[o] &: \text{tiempo utilizado por el operario } o, \\ comp[ot] &: \text{tipos de tareas ya completadas en la OT,} \\ occ[ot] &: \text{intervalos ocupados por la OT.} \end{aligned}$$

4.2.3. Representación de operarios, tareas y repuestos

- Las tareas se representan por su índice i y atributos: tipo, orden y duración.
- Un operario sólo puede ejecutar tareas cuyos tipos estén en $E(t)$.
- Los repuestos se representan mediante el vector binario R_{ot} .

4.2.4. Tiempos de operación y estructura temporal

El tiempo se representa en minutos dentro del intervalo $[0, H]$. Una tarea i ejecutada por un operario o genera un intervalo:

$$[s_i, f_i) = [tpo[o], tpo[o] + p(tipo(i))).$$

4.2.5. Restricciones formales del taller

Una tarea i puede ejecutarse si y sólo si:

- (1) $o \in E(tipo(i))$ Aptitud del operario,
- (2) $R_{ot(i)}$ provee repuesto disponible Repuesto disponible,
- (3) $P(tipo(i)) \cap T_{ot(i)} \subseteq comp[ot(i)]$ Precedencias cumplidas,
- (4) $f_i \leq H$ No excede jornada,
- (5) $[s_i, f_i) \cap occ[ot(i)] = \emptyset$ No solapa con tareas de la misma OT.

4.3. Construcción del generador de instancias

4.3.1. Parámetros de la simulación y alcance

Cada día se genera una instancia compuesta por:

- Un conjunto de órdenes.
- Un conjunto de tareas concretas.
- Habilidades del personal.
- Duraciones de tareas.
- Precedencias.
- Repuestos disponibles.

4.3.2. Modelo de aleatoriedad y distribución de tiempos

El generador recibe:

- El número de órdenes,
- El número de tareas por orden,
- Su distribución temporal (según $p(t)$),
- Disponibilidad de repuestos,
- Habilidades por tipo.

4.3.3. Procedimiento de creación de escenarios

El proceso consiste en:

1. Seleccionar órdenes y sus tareas.
2. Asignar tipos y duraciones.
3. Generar disponibilidad de repuestos.
4. Asignar operarios aptos por tipo.
5. Construir la instancia final (OT, T_d, O) .

4.3.4. Validación del modelo simulado con expertos

Los parámetros fueron validados con personal del taller, asegurando que:

- Las duraciones son realistas.
- Las precedencias corresponden al flujo mecánico real.
- Las habilidades reflejan la estructura del personal.
- El horizonte de 8 horas es consistente con el turno laboral.

4.3.5. Pseudocódigo del simulador operativo

El corazón de la simulación consiste en determinar si una asignación propuesta es válida. El pseudocódigo siguiente refleja el comportamiento operativo del taller:

Algorithm 1: Simulación operativa de programación de tareas

```

Inicializar estructuras:  $tpo[o] \leftarrow 0$ ,  $comp[ot] \leftarrow \emptyset$ ,  $occ[ot] \leftarrow \emptyset$ ;
foreach operario  $o \in O$  do
    Obtener conjunto de tareas asignadas  $T_o$ ;
    Ordenar  $T_o$  según prioridad descendente;
    foreach tarea  $i \in T_o$  do
        Verificar aptitud:  $o \in E(tipo(i))$ ;
        Verificar repuesto disponible;
        Verificar precedencias cumplidas en  $comp[ot(i)]$ ;
        Calcular intervalo  $[s_i, f_i] = [tpo[o], tpo[o] + p(tipo(i))]$ ;
        if  $f_i > H$  then
            | rechazar tarea; continuar
        end
        if  $[s_i, f_i]$  solapa con algún intervalo en  $occ[ot(i)]$  then
            | rechazar tarea; continuar
        end
        Aceptar tarea: actualizar  $tpo[o]$ ,  $comp$ ,  $occ$ ;
    end
end

```

4.4. Conjunto final de escenarios utilizados en la comparación

4.4.1. Número y estructura de las instancias

Para la fase experimental se consideraron múltiples instancias generadas bajo diferentes configuraciones de:

- número de órdenes,
- número de tareas,
- complejidad de precedencias,

- disponibilidad de recursos.

4.4.2. Variabilidad incorporada

Las instancias presentan variabilidad en:

- tamaño del conjunto de tareas,
- carga operativa,
- composición de órdenes,
- mezcla de repuestos disponibles.

4.4.3. Justificación de los casos de estudio

Estas instancias permiten evaluar el desempeño del método SPT y del algoritmo genético en condiciones diversas, representando escenarios realistas del taller.

Capítulo 5

Desarrollo de la Solución

La asignación de tareas en talleres industriales suele realizarse mediante procesos manuales que dependen de la experiencia del operario y de la disponibilidad momentánea de recursos. Este enfoque puede generar sobrecarga de trabajo, tiempos inactivos, secuencias operativas subóptimas y retrasos en la entrega de órdenes, especialmente cuando cada tarea requiere habilidades técnicas específicas, disponibilidad de repuestos y una coordinación adecuada con otras actividades del proceso de reparación. El problema resultante puede entenderse como una variante del Job Shop Scheduling con restricciones adicionales, como precedencias internas por orden de trabajo, diferencias en la aptitud de los operarios y un horizonte diario limitado. Esta combinación de factores da lugar a un espacio de búsqueda complejo y no lineal, típico de problemas NP-hard, lo que dificulta resolverlo mediante métodos exactos en tiempos razonables.

5.1. Enfoque de Solución: Algoritmos Genéticos (AG)

Frente a la complejidad operativa descrita, se implementa un enfoque de optimización basado en Algoritmos Genéticos (AG) cuya configuración se adapta específicamente al flujo de trabajo del taller. Este enfoque permite representar simultáneamente la asignación de operarios y la secuenciación de las tareas de cada orden de trabajo, incorporando las restricciones reales del sistema —habilidades requeridas, disponibilidad de insumos, dependencias entre actividades y límite diario de jornada— dentro del proceso evolutivo.

De esta manera, el AG actúa como un mecanismo que genera y mejora iterativamente configuraciones de programación factibles y alineadas con las condiciones operativas del entorno industrial.

5.1.1. Modelado del Problema para el Algoritmo Genético

El algoritmo genético opera sobre la estructura formal del problema definida en el capítulo anterior, utilizando como insumo el conjunto diario de órdenes de trabajo, tareas concretas, operarios y parámetros operativos del taller. Sobre esta base, el AG debe construir una solución que represente un cronograma factible, respetando las restricciones del sistema y maximizando la cantidad de tareas ejecutadas dentro de la jornada laboral.

En este modelo, cada solución candidata corresponde a una propuesta completa de asignación y secuenciación de las tareas del día. Para ello, el AG toma dos decisiones fundamentales: (1) determinar a qué operario asignar cada tarea y (2) definir el orden en que cada tarea debe intentarse programar dentro del flujo operativo del día.

Estas decisiones se construyen a partir de los elementos establecidos en la formalización del taller: los conjuntos de tareas T_d , órdenes OT y operarios O ; los conjuntos de aptitud $E(t)$ por tipo de tarea; las precedencias internas $P(t)$; la disponibilidad de repuestos por orden; y el horizonte diario de 480 minutos para cada operario.

Durante el proceso de evaluación, la función de aptitud (fitness) utiliza esta información para determinar la viabilidad de la solución propuesta. Para ello actualiza variables como el tiempo acumulado por operario, las tareas completadas dentro de cada orden y los intervalos programados, verificando que no existan solapamientos y que todas las restricciones sean satisfechas o penalizadas adecuadamente.

De esta manera, el modelado del problema proporciona el marco estructural que guía el comportamiento del AG, permitiéndole explorar configuraciones viables dentro de un espacio de búsqueda altamente restringido.

5.1.2. Representación del Individuo

La solución que manipula el algoritmo genético se estructura mediante un cromosoma que integra explícitamente las dos decisiones fundamentales del

problema: la asignación de operarios y el orden de intento de programación de cada tarea. Para ello, cada individuo se modela como un vector de dimensión $2n$, donde n corresponde al número de tareas del día.

$$\mathbf{x} = [y_1, y_2, \dots, y_n \mid \pi_1, \pi_2, \dots, \pi_n]$$

donde:

$$y_i \in E(\text{tipo}(i)), \quad \pi_i \in [0, 1], \quad i = 1, \dots, n.$$

La primera mitad del cromosoma contiene la asignación propuesta para cada tarea. El gen y_i especifica el operario responsable de ejecutar la tarea i , y su valor está restringido al conjunto de operarios aptos $E(\text{tipo}(i))$, tal como se definió en la formalización del problema. Con esto, la representación inicial garantiza que cada asignación respete las habilidades mínimas requeridas por el taller.

La segunda mitad del cromosoma codifica la prioridad asociada a cada tarea mediante un valor continuo $\pi_i \in [0, 1]$. Estas prioridades determinan el orden en que las tareas serán consideradas durante el proceso de evaluación, orientando la construcción del flujo operativo. Este mecanismo permite explorar distintas secuencias de programación sin necesidad de trabajar directamente con permutaciones, lo cual simplifica el espacio de búsqueda y facilita la integración con las restricciones industriales del sistema.

La combinación de ambas partes —asignación y prioridad— permite que cada individuo represente un cronograma candidato completo, el cual será posteriormente evaluado mediante la función de aptitud. Esta estructura dual ofrece flexibilidad para adaptarse a las condiciones reales del taller, manteniendo al mismo tiempo una codificación compacta y eficiente para la evolución genética.

5.1.3. Evaluación de la Solución

Una vez representado el individuo mediante la asignación de operarios y las prioridades de programación, el siguiente paso consiste en evaluar su desempeño como cronograma operativo. Este proceso corresponde a la simulación interna que ejecuta el algoritmo genético para determinar qué tan viable y eficiente es la solución candidata.

La evaluación parte de separar las tareas asignadas a cada operario a partir del vector $\mathbf{y}$. Posteriormente, cada conjunto T_o se ordena según las prioridades

π_i , definiendo así el orden en que las tareas serán consideradas. Para cada tarea, el proceso verifica secuencialmente las restricciones operativas del taller: aptitud del operario, disponibilidad de repuestos, cumplimiento de precedencias internas dentro de la orden de trabajo y ausencia de solapamientos en los intervalos ya programados. Si la tarea satisface dichas condiciones y su tiempo de finalización no excede el horizonte diario del operario, se programa y se actualizan las estructuras internas de la evaluación. En caso contrario, se registra la penalización correspondiente.

Al finalizar el recorrido, la evaluación produce un conjunto de métricas que describen el comportamiento del cronograma: número de tareas ejecutadas, violaciones de precedencia, faltas de repuestos, asignaciones no aptas, solapamientos internos, excedentes del horizonte, balance de carga y tiempo máximo de finalización. Estas métricas alimentan posteriormente la función de aptitud.

Representación Formal del Proceso de Evaluación. La evaluación del individuo sigue un procedimiento estructurado en seis pasos principales, como se describe a continuación:

1. Separación por operario. Dado un individuo $\mathbf{x} = (\mathbf{y}, \boldsymbol{\pi})$, se construye el conjunto de tareas asignadas a cada operario:

$$T_o = \{i \in T_d : y_i = o\} \quad \forall o \in O$$

Cada conjunto T_o contiene las tareas asignadas al operario o según el vector de asignación $\mathbf{y}$.

2. Ordenamiento por prioridad. Para cada operario, se ordena el conjunto T_o según los valores de prioridad π_i contenidos en la segunda mitad del cromosoma:

$$\text{Orden}_o = \text{sort}(T_o, \text{clave} = \pi_i)$$

donde el operador sort organiza las tareas en orden ascendente (o descendente, según la implementación) de prioridad. Esto define el orden en que cada tarea será considerada para su programación.

3. Intento de programación. Sea s_i el inicio tentativo y f_i el fin tentativo de la tarea i . Se asigna:

$$s_i = tpo[o], \quad f_i = s_i + p(i)$$

donde $tpo[o]$ es el tiempo acumulado (tiempo posterior ocupado) del operario o , y $p(i)$ es la duración de la tarea i .

4. Validación de restricciones. Antes de confirmar la programación, se verifica que se cumplan:

1. **Aptitud:** $y_i \in E(\text{tipo}(i))$
2. **Disponibilidad de repuesto:** $\text{rep}(OT(i)) = \text{disponible}$
3. **Precedencias:** $\forall t' \in P(i) : t' \in \text{comp}[OT(i)]$
4. **Posible ubicación en agenda:** $s_i = tpo[o], f_i = s_i + p(i)$
5. **No solapamiento dentro de la misma orden:** $\forall [s', f'] \in \text{occ}[OT(i)] : \neg(s_i < f' \wedge f_i > s')$
6. **Horizonte diario:** $f_i \leq H$

Si todas las condiciones se satisfacen, la tarea se programa y se registra su intervalo $[s_i, f_i]$ en $\text{occ}[OT(i)]$. En caso contrario, se registra la penalización correspondiente.

Actualización de estado. Si la programación es exitosa:

$$\text{programar}(i, o), \quad tpo[o] \leftarrow f_i, \quad \text{comp}[OT(i)] \leftarrow \text{comp}[OT(i)] \cup \{i\}$$

Se actualiza el tiempo acumulado del operario y el conjunto de tareas completadas de la orden.

Registro de penalización. En caso contrario, se registra:

registrar penalización correspondiente

Este proceso se repite para cada tarea en Orden_o de cada operario $o \in O$.

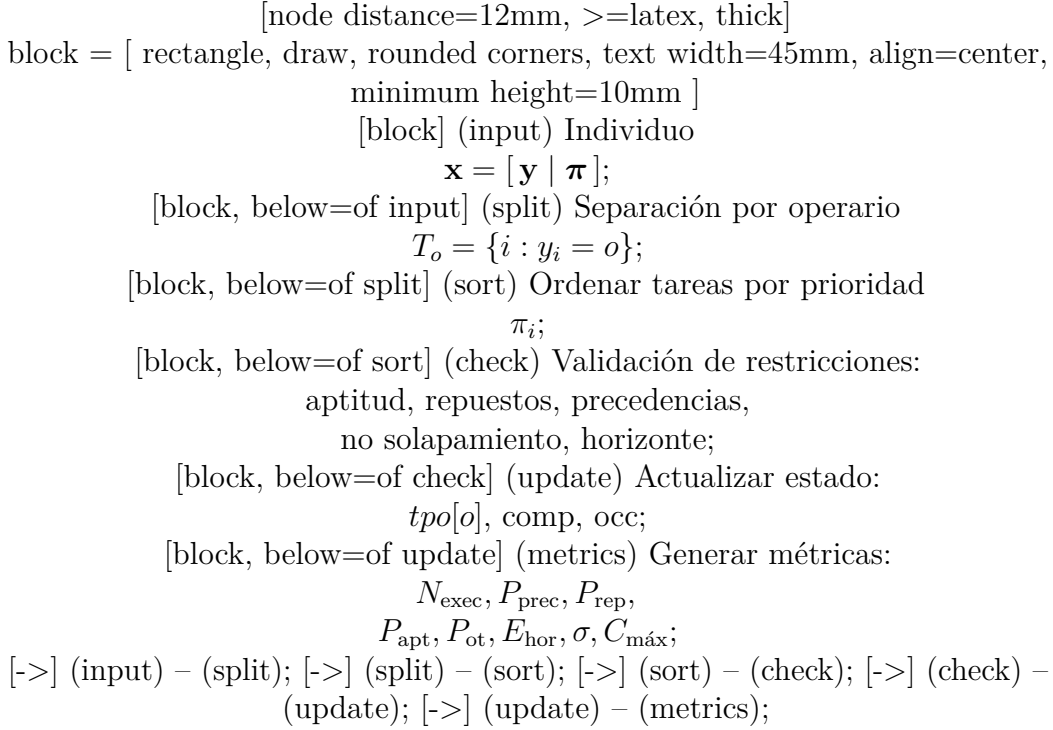


Figura 5.1: Flujo del proceso de evaluación de un individuo.

5.1.4. Función de Aptitud

La función de aptitud asigna un valor cuantitativo al desempeño de cada individuo a partir de las métricas generadas durante la evaluación. Su diseño busca orientar el proceso evolutivo hacia soluciones que maximicen la cantidad de tareas programadas y minimicen las violaciones operativas. Para ello se emplean ponderaciones que reflejan la importancia relativa de cada componente dentro del contexto del taller.

El término principal corresponde al número de tareas ejecutadas, ponderado con un valor significativamente mayor, dado que la productividad diaria es el objetivo fundamental del proceso de programación. Las penalizaciones

restantes representan violaciones a restricciones que afectan la factibilidad o la calidad de la programación. Estas penalizaciones se escalan según su impacto operativo: las restricciones relacionadas con aptitud, precedencias y repuestos reciben pesos más altos, ya que su incumplimiento impide la ejecución de la tarea. Los solapamientos dentro de una orden se penalizan con magnitud similar, pues afectan la coherencia del proceso de reparación. Por su parte, los excedentes del horizonte, el balance de carga y el makespan tienen ponderaciones menores, dado que no invalidan la tarea pero sí influyen en la eficiencia global del cronograma.

La función de aptitud se define formalmente como:

$$\text{fitness}(\mathbf{x}) = 100 \cdot N_{\text{exec}} - 10 \cdot P_{\text{prec}} - 10 \cdot P_{\text{rep}} - 15 \cdot P_{\text{apt}} - 12 \cdot P_{\text{ot}} - 0,1 \cdot E_{\text{hor}} - 15 \cdot \sigma - 10 \cdot C_{\text{máx}}$$

donde:

- N_{exec} : número de tareas ejecutadas satisfactoriamente.
- P_{prec} : penalización acumulada por violaciones de precedencias.
- P_{rep} : penalización acumulada por falta de repuestos disponibles.
- P_{apt} : penalización acumulada por asignaciones a operarios no aptos.
- P_{ot} : penalización acumulada por solapamientos dentro de órdenes de trabajo.
- E_{hor} : excedente total del horizonte diario acumulado.
- σ : desviación estándar de la carga de trabajo entre operarios (desbalance).
- $C_{\text{máx}}$: makespan o tiempo máximo de finalización del cronograma.

Los coeficientes responden a los siguientes criterios operativos:

- **100 en N_{exec}** : maximizar la ejecución de tareas es el objetivo operativo principal. Cada tarea adicional contribuye significativamente a la aptitud.
- **15 en P_{apt}** : una asignación a un operario no apto hace imposible la ejecución de la tarea, por lo que recibe la penalización más alta entre restricciones.

- **12 en P_{ot} :** los solapamientos dentro de una orden de trabajo comprometen la coherencia del proceso de reparación.
- **10 en P_{prec} y P_{rep} :** la falta de precedencias o repuestos impide ejecutar la tarea, recibiendo penalizaciones significativas.
- **0.1 en E_{hor} :** el excedente del horizonte indica ineficiencia temporal pero no invalida la tarea, recibiendo baja penalización.
- **15 en σ :** el desbalance de carga se penaliza fuertemente para promover una distribución equitativa de trabajo entre operarios.
- **10 en $C_{\text{máx}}$:** favorece cronogramas más compactos, reduciendo el tiempo total de ejecución.

En conjunto, estas ponderaciones guían la búsqueda del algoritmo genético hacia soluciones factibles, eficientes y alineadas con las necesidades reales del entorno industrial.

5.1.5. Operadores Genéticos y Parámetros del AG

El proceso evolutivo implementado en este proyecto se desarrolla mediante los operadores genéticos estándar proporcionados por la librería PyGAD, configurados de manera que reflejen las características del problema y favorezcan una búsqueda eficiente dentro del espacio de soluciones. Cada operador contribuye a la exploración o explotación del espacio, mientras que los parámetros controlan la estabilidad y la velocidad de convergencia del algoritmo.

Selección. Se emplea selección por *ranking*, mecanismo en el cual los individuos se ordenan de acuerdo con su aptitud y se asignan probabilidades de selección según su posición relativa. Este método es adecuado en contextos donde las diferencias entre aptitudes pueden ser grandes debido a penalizaciones por restricciones, evitando que unos pocos individuos dominen prematuramente la población y manteniendo una presión selectiva estable.

Cruce. Para la recombinación se utilizó un cruce de un solo punto (*single point crossover*). Este método divide el cromosoma en dos segmentos y combina parcialmente la asignación de operarios con el vector de prioridades proveniente de otro progenitor. Dado que el cromosoma está estructurado en

dos bloques homogéneos (asignación y prioridad), un punto de cruce introduce variabilidad sin descomponer significativamente la estructura semántica del individuo.

Mutación. La variación genética se incorpora mediante una tasa de mutación aplicada al 15 % de los genes del cromosoma. En los genes de asignación, la mutación consiste en seleccionar aleatoriamente un operario válido para el tipo de tarea; en los genes de prioridad, asigna un valor continuo aleatorio en el intervalo $[0, 1]$. Esta tasa moderada permite explorar nuevas combinaciones manteniendo la estabilidad del proceso evolutivo y evitando convergencias prematuras.

Parámetros generales. El algoritmo se ejecutó con los siguientes parámetros:

- **Población:** 100 individuos por generación.
- **Generaciones:** 300 ciclos evolutivos.
- **Padres seleccionados por generación:** 10 individuos.
- **Padres preservados (elitismo):** 5 mejores individuos.
- **Tipo de selección:** *ranking*, adecuado para distribuciones de aptitud desiguales.
- **Operador de cruce:** *single point*, divide el cromosoma en dos segmentos.
- **Tipo de mutación:** *random*.
- **Probabilidad de mutación:** 5 % de los genes mutados por generación.
- **Semilla aleatoria:** 42, para garantizar reproducibilidad de resultados.

Estos parámetros fueron seleccionados mediante validación empírica previa y se alinean con estándares de la literatura de algoritmos genéticos. La configuración proporciona un equilibrio adecuado entre exploración del espacio de búsqueda (favorecida por la mutación y la selección por ranking) y refinamiento de soluciones prometedoras (garantizado por el elitismo y la

población moderada). La semilla fija permite reproducir exactamente los mismos resultados en futuras ejecuciones del algoritmo, facilitando la validación y comparación de resultados.

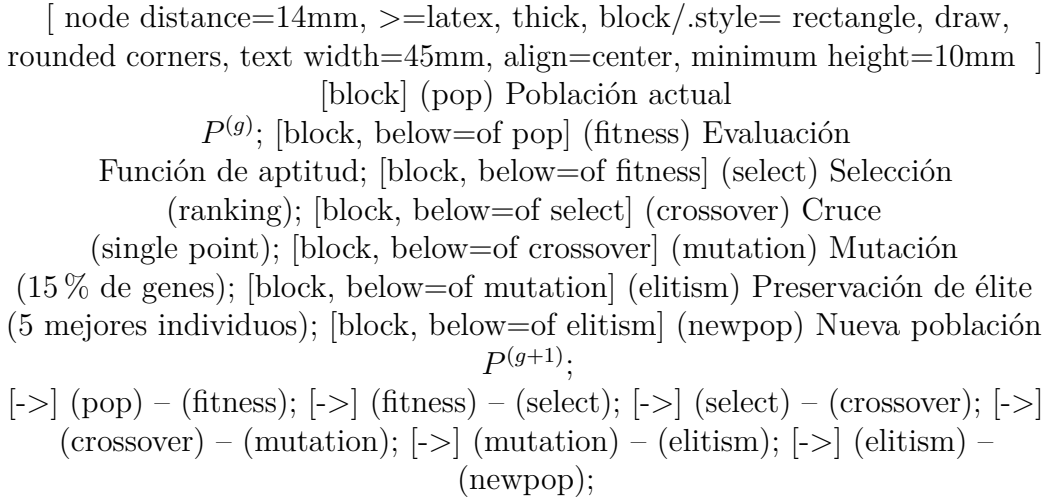


Figura 5.2: Flujo del ciclo evolutivo del algoritmo genético en PyGAD.

5.2. Enfoque de Solución: Método SPT

Como alternativa al enfoque evolutivo basado en algoritmos genéticos, se consideró una estrategia heurística clásica de secuenciación: la regla SPT (*Shortest Processing Time*). Este método es ampliamente utilizado en entornos industriales debido a su simplicidad, su bajo costo computacional y su buen desempeño en escenarios donde las restricciones operativas son mínimas. Por estas razones, SPT constituye un punto de referencia habitual en estudios comparativos de métodos de programación.

En su forma tradicional, SPT prioriza las tareas con menor tiempo de procesamiento. No obstante, dado que el problema del taller incorpora múltiples restricciones —precedencias internas, disponibilidad de repuestos, aptitud del operario y un horizonte diario limitado—, la regla fue adaptada para integrarse al entorno de evaluación del sistema.

De este modo, tanto SPT como el algoritmo genético abordan el mismo problema de programación y son plenamente capaces de generar un cronograma operativo completo. La diferencia radica en la lógica que guía la

construcción de la solución: mientras el enfoque evolutivo explora múltiples configuraciones mediante un proceso de búsqueda global, SPT define un orden determinista basado en la duración de las tareas.

5.2.1. Modelado del SPT para el Taller

La aplicación de la regla SPT en el taller requiere establecer con claridad la información operativa que interviene en la selección y ordenamiento de las tareas. En este enfoque, SPT actúa directamente sobre los datos de la instancia, priorizando las tareas en función de su tiempo de procesamiento, pero integrado a las restricciones reales del sistema para asegurar un orden viable.

El método utiliza como entrada las tareas pendientes por orden de trabajo, los tiempos de procesamiento por tipo de tarea, las relaciones de precedencia interna, la disponibilidad de repuestos, el conjunto de operarios disponibles y las tareas para las cuales son aptos, así como el horizonte diario máximo permitido.

Con esta información, el algoritmo identifica en cada iteración las tareas listas para ser consideradas. Una tarea se considera lista cuando: (i) todas las tareas precedentes dentro de su orden han sido completadas, (ii) el repuesto correspondiente se encuentra disponible, (iii) su duración está definida y (iv) permanece pendiente en la instancia.

Entre todas las tareas que cumplen estas condiciones, la regla SPT selecciona aquellas con menor tiempo de procesamiento, conformando un conjunto ordenado dinámicamente. Este ordenamiento se actualiza a medida que nuevas tareas se completan, repuestos se consumen y las relaciones de precedencia modifican el estado del sistema.

El resultado de este proceso es una política determinista que, basada en la duración de las tareas y en el cumplimiento de las restricciones operativas, establece un orden de ejecución coherente con las características del taller. A partir de dicho orden, el entorno de programación determina los horarios de inicio y finalización, respetando los tiempos disponibles, la aptitud del operario asignado y las ventanas temporales de la orden de trabajo.

5.2.2. Representación Formal del Orden SPT

El ordenamiento SPT aplicado al taller puede describirse formalmente como un proceso iterativo que opera sobre el conjunto de tareas pendientes.

En cada paso, el método determina el subconjunto de tareas que cumplen las condiciones necesarias para ser consideradas y luego aplica un criterio de priorización basado en el tiempo de procesamiento.

Sea $\mathcal{T}$ el conjunto de tareas pendientes, $p(i)$ la duración de la tarea i , $\mathcal{P}(ot, i)$ el conjunto de prerequisites internos para la tarea i dentro de la orden ot , $R(ot, i)$ un indicador de disponibilidad del repuesto correspondiente, y $C(ot)$ el conjunto de tareas completadas en dicha orden. El conjunto de tareas listas en la iteración k se define como:

$$\mathcal{L}^{(k)} = \left\{ (ot, i) \in \mathcal{T}^{(k)} : \mathcal{P}(ot, i) \subseteq C^{(k)}(ot), R(ot, i) = 1, p(i) \text{ definido} \right\}.$$

Sobre este conjunto, el criterio SPT establece un ordenamiento ascendente por duración:

$$\text{SPT}(\mathcal{L}^{(k)}) = \text{sort} \left(\mathcal{L}^{(k)}, p(i) \right).$$

Dado que los elementos dentro de una misma orden pueden compartir la misma duración, se emplean criterios secundarios de desempate consistentes con la implementación: primero por identificador de orden y luego por identificador de tarea. Esto asegura un orden total determinista:

$$(ot_a, i_a) \prec (ot_b, i_b) \iff \begin{cases} p(i_a) < p(i_b), \\ \text{o bien } p(i_a) = p(i_b) \text{ y } ot_a < ot_b, \\ \text{o bien } p(i_a) = p(i_b), ot_a = ot_b \text{ y } i_a < i_b. \end{cases}$$

El resultado final es una secuencia ordenada de tareas listas, actualizada dinámicamente en cada iteración a medida que nuevas tareas son completadas y otras entran en condición de ser consideradas.

5.2.3. Aplicación Operativa del SPT en el Taller

Una vez definido el orden SPT de las tareas listas, el sistema procede a construir el cronograma operativo respetando la disponibilidad de los operarios, las restricciones de cada orden de trabajo y el horizonte diario. Este proceso se desarrolla de manera iterativa sobre el conjunto de tareas pendientes, actualizando el estado del sistema a medida que se programan nuevas actividades.

En cada iteración, el método toma la lista ordenada de tareas generada por la regla SPT y selecciona, para cada una de ellas, un operario apto para su ejecución. La asignación se realiza siguiendo un criterio de carga acumulada: entre los operarios capaces de ejecutar la tarea, se elige aquel que tenga el menor tiempo total programado hasta el momento, buscando equilibrar progresivamente la utilización de la capacidad diaria.

Una vez identificado el operario, el sistema determina el primer intervalo disponible donde la tarea puede ser ubicada. Para ello se utiliza una rutina de búsqueda del hueco más temprano, que evalúa: (i) los intervalos ya ocupados por actividades de la misma orden de trabajo, (ii) el tiempo más próximo en el que el operario puede iniciar nuevas tareas, (iii) la duración de la actividad y (iv) el límite superior del horizonte diario.

Si existe un intervalo $[s, f)$ dentro de estos límites donde la tarea puede ejecutarse sin solaparse con otras actividades de la misma orden y sin exceder la jornada asignada, la tarea es programada en dicho espacio. Con esto se actualizan simultáneamente la ocupación temporal de la orden, el tiempo acumulado del operario y el conjunto de tareas completadas.

En caso de que ninguna combinación válida de operario e intervalo esté disponible, la tarea permanece pendiente y será reconsiderada más adelante, siempre que las condiciones del sistema cambien a su favor. El ciclo continúa hasta que no sea posible programar nuevas tareas o todas hayan sido ubicadas dentro del horizonte.

Al finalizar, el método genera el cronograma de tareas con tiempos de inicio y fin, la lista de tareas que no pudieron ser programadas, la ocupación temporal por orden de trabajo, el tiempo total utilizado por cada operario y métricas como el número de tareas ejecutadas y el makespan.

De esta manera, el enfoque SPT produce un cronograma completo y coherente con las restricciones del taller, combinando un orden de secuenciación basado en los tiempos de procesamiento con un proceso operativo de programación ajustado al estado real del sistema.

[node distance=12mm, >=latex, thick, block/.style= rectangle, draw, rounded corners, text width=40mm, align=center, minimum height=10mm]

[block] (pend) Tareas pendientes; [block, below=of pend] (filtrar) Filtrado de tareas listas

(prereqs + repuestos); [block, below=of filtrar] (orden) Orden SPT (por duración); [block, below=of orden] (operario) Selección de operario apto (mínima carga); [block, below=of operario] (slot) Búsqueda de hueco más temprano; [block, below=of slot] (asignar) Asignación inicio-fin; [block, below=of asignar] (actualizar) Actualización del estado (pendientes, carga, OT);

[->] (pend) – (filtrar); [->] (filtrar) – (orden); [->] (orden) – (operario); [->] (operario) – (slot); [->] (slot) – (asignar); [->] (asignar) – (actualizar);

Figura 5.3: Flujo operativo del método SPT en el taller.

Capítulo 6

Ejecución y Obtención de Resultados

6.1. Entorno de desarrollo

El sistema fue implementado en el lenguaje de programación Python, debido a su flexibilidad, amplia disponibilidad de librerías para optimización y manejo de datos, y su compatibilidad con métodos heurísticos y evolutivos. Para garantizar portabilidad y consistencia, se empleó un entorno virtual administrado mediante **venv**.

Las principales dependencias utilizadas fueron:

- **NumPy**: manejo eficiente de arreglos y operaciones matemáticas.
- **Pandas**: construcción de estructuras tabulares para generar y almacenar instancias.
- **PyGAD**: librería empleada para la implementación del algoritmo genético.
- **Random**: generación de valores pseudoaleatorios para la creación de instancias.

El archivo **requirements.txt** incluye las versiones exactas utilizadas, permitiendo reproducir las condiciones de ejecución en cualquier sistema.

6.2. Diseño del software

El proyecto se estructuró siguiendo una arquitectura modular, organizada en tres componentes principales: generación de instancias, algoritmos de programación (AG y SPT) y simulación. Esto permite separar claramente la lógica del modelo del taller, los métodos de asignación y el procedimiento experimental.

6.2.1. Arquitectura general del sistema

A continuación se presenta la organización del software:

- **config/**: contiene la configuración del taller y el generador de instancias.
- **ag/**: módulo que implementa el algoritmo genético.
- **spt/**: módulo con la implementación del método SPT.
- **simulador.py**: orquesta la ejecución de instancias, aplicar ambos algoritmos y almacenar los resultados.

Cada componente se comunica únicamente mediante estructuras de datos bien definidas, lo que facilita la extensibilidad y la validación del sistema.

6.2.2. Flujo general del software

El funcionamiento completo del sistema puede resumirse como:

1. **Generación de instancias**: se construyen escenarios según la configuración del taller.
2. **Ejecución de SPT y AG**: cada instancia se evalúa mediante ambos métodos.
3. **Simulación operativa**: se usa la simulación formalizada en el Capítulo 4 para determinar las tareas aceptadas, rechazadas y métricas de desempeño.
4. **Consolidación de resultados**: se almacenan métricas en archivos CSV para su análisis posterior.

Este pipeline es consistente con el objetivo principal de la tesis: comparar el desempeño del algoritmo genético frente a un método clásico de ingeniería industrial.

6.3. Implementación del algoritmo genético

El módulo `ag/ag.py` implementa el algoritmo genético utilizando la librería PyGAD. El enfoque se basa en la representación dual del individuo, donde cada solución codifica simultáneamente la asignación de operarios y la prioridad de ejecución de cada tarea.

6.3.1. Representación del individuo

Dado un conjunto de tareas del día con tamaño n , se construye un cromosoma de longitud $2n$:

$$\text{individuo} = [o_1, \dots, o_n, w_1, \dots, w_n]$$

donde:

- o_i es el operario seleccionado para la tarea i .
- w_i es un valor que representa la prioridad relativa de la tarea.

Esta representación permite modular tanto la asignación como la secuencia interna de ejecución.

6.3.2. Generación del espacio de búsqueda

El archivo `taller_config.py` define, para cada tipo de tarea, qué operarios son aptos para ejecutarla. Esta información se utiliza para construir dinámicamente el *gene space* del AG, restringiendo valores inválidos y reduciendo el espacio de búsqueda.

6.3.3. Función de aptitud

La función de aptitud está implementada de manera que evalúa cada individuo llamando al simulador operativo descrito en el Capítulo 4. El resultado final se obtiene mediante una métrica compuesta que penaliza:

- Tareas rechazadas.
- Precedencias incumplidas.
- Exceso de carga.

Y recompensa:

- Cantidad de tareas ejecutadas.
- Balance de carga entre operarios.

El pseudocódigo general de la evaluación es:

Algorithm 2: Evaluación de un individuo mediante simulación operativa

Obtener asignación de operarios y prioridades desde el individuo;
Construir ordenamiento de tareas según prioridades;
Aplicar simulación operativa;
Calcular métricas: ejecutadas, rechazadas, makespan, carga total, etc.;
Combinar métricas en una aptitud final;

6.3.4. Integración con PyGAD

El AG se ejecuta definiendo:

- tamaño de población,
- número de generaciones,
- probabilidad de cruce,
- probabilidad de mutación.

PyGAD administra el proceso evolutivo, mientras que la evaluación real se realiza con el simulador del taller.

6.3.5. Configuración base del algoritmo genético

Antes de iniciar el proceso de calibración, se definió una configuración base del algoritmo genético que sirvió como punto de partida para todos los experimentos. Esta configuración incluye parámetros estándar recomendados en la literatura y valores validados empíricamente para garantizar un comportamiento estable del algoritmo.

- Tamaño de población: 100 individuos
- Número de padres por generación: 10
- Padres preservados: 5
- Método de selección: *rank*
- Operador de cruce: *single_point*
- Probabilidad de mutación: 15 %
- Tipo de mutación: *random*
- Semilla aleatoria: 42

Esta configuración sirvió como referencia para evaluar de forma independiente cada parámetro crítico del algoritmo genético (operador de cruce, número de generaciones y probabilidad de mutación), tal como se detalla en las secciones siguientes.

6.4. Implementación del método SPT

El módulo `spt/spt.py` implementa la heurística Shortest Processing Time (SPT), adaptada al contexto del taller. La versión utilizada en esta tesis mantiene el espíritu del método clásico, pero incorpora restricciones reales del taller, como habilidades de operarios, repuestos y precedencias.

6.4.1. Ordenamiento de tareas

El método ordena las tareas según la duración:

$$\text{orden SPT : } p(\text{tipo}(i_1)) \leq p(\text{tipo}(i_2)) \leq \dots$$

Este orden se ajusta automáticamente según precedencias y recursos disponibles.

6.4.2. Asignación de operarios y validación

Para cada tarea ordenada, el método:

1. identifica operarios aptos,
2. selecciona aquel con menor carga acumulada,
3. verifica disponibilidad de repuestos,
4. evalúa posibles solapamientos dentro de la misma OT.

Si la tarea no puede ejecutarse bajo las restricciones del taller, se marca como rechazada.

6.5. Ejecución de simulaciones

El archivo `simulador.py` ejecuta el experimento completo. Para cada instancia generada, se aplican consecutivamente el método SPT y el algoritmo genético, almacenando métricas clave que permitirán una comparación justa.

6.5.1. Generación de conjuntos de instancias

Las instancias se generan con el módulo `generador_instancias.py`, que permite configurar:

- número mínimo y máximo de órdenes,
- número total de tareas,
- distribución de tipos,
- disponibilidad de repuestos.

6.5.2. Ejecución conjunta de SPT y AG

Para cada instancia:

1. se ejecuta SPT y se registra su desempeño,
2. se ejecuta el AG y se obtiene el mejor individuo,
3. ambos resultados se evalúan con el simulador,
4. se registran métricas comparables.

6.5.3. Consolidación de resultados

Los datos generados por cada ejecución se almacenan en un archivo CSV, que posteriormente es utilizado en el Capítulo 7 para el análisis cuantitativo.

6.5.4. Reproducibilidad

Para garantizar repetibilidad, se fijan semillas aleatorias y se documentan los parámetros utilizados en cada corrida experimental.

6.5.5. Evaluación del Operador de Cruce

Con el objetivo de determinar el operador de cruce más adecuado para el algoritmo genético, se llevó a cabo un experimento comparativo empleando los dos operadores disponibles en la librería PyGAD que preservan parcialmente la estructura del individuo: *single_point* y *two_points*. Ambos operadores fueron evaluados bajo exactamente las mismas condiciones: misma población, mismas instancias, misma semilla aleatoria y la configuración base descrita en la Sección ??.

El experimento se ejecutó sobre un conjunto de veinte instancias independientes, todas simuladas bajo un horizonte operativo de 480 minutos (equivalente a una jornada laboral completa). Este horizonte amplio permite observar diferencias reales en el comportamiento evolutivo; horizontes más cortos generan dinámicas artificialmente similares y no son adecuados para la calibración del algoritmo genético.

Los resultados promedio obtenidos para cada operador se presentan en la Tabla ??.

Tabla 6.1: Resultados promedio del AG para cada operador de cruce (horizonte de 480 minutos).

Operador de cruce	Tareas ejecutadas	Tareas rechazadas	Makespan	Carga total
<i>single_point</i>	26.20	17.55	212.60	613.85
<i>two_points</i>	24.55	14.85	206.10	568.80

Los resultados evidencian diferencias consistentes entre los dos operadores evaluados. Aunque *two_points* obtiene un makespan ligeramente menor, el operador *single_point* supera de manera sistemática a su contraparte en las métricas más relevantes para el problema: la cantidad de tareas ejecutadas y la carga total. En promedio, *single_point* logra programar 1.65 tareas adicionales por instancia y utiliza de manera más intensiva el tiempo disponible de los operarios, lo cual es crítico dado que el objetivo principal del sistema es maximizar la cantidad de trabajo completado durante la jornada.

La mayor cantidad de tareas ejecutadas está alineada con la función de aptitud del AG, que prioriza explícitamente este criterio. El mejor comportamiento observado para *single_point* indica que este operador favorece combinaciones más estables entre asignación y prioridad, preservando estructuras útiles a lo largo del proceso evolutivo.

En conclusión, con base en la evidencia experimental, se seleccionó el operador *single_point* como el más adecuado y este fue utilizado en los experimentos posteriores del proceso de calibración del algoritmo genético.

6.5.6. Evaluación del Número de Generaciones

Una vez seleccionado el operador de cruce *single_point*, el siguiente paso en la calibración del algoritmo genético consistió en determinar un número adecuado de generaciones. Este parámetro controla la profundidad de la búsqueda evolutiva y, por tanto, influye directamente en la calidad de las soluciones obtenidas y en el costo computacional del proceso.

Siguiendo recomendaciones comunes en la literatura sobre algoritmos genéticos, se evaluaron tres niveles de profundidad evolutiva: 300, 800 y 1300 generaciones. Estos valores representan una escala progresiva de exploración del espacio de búsqueda (baja, media y alta), permitiendo observar cómo varía el desempeño del AG a medida que se incrementa el esfuerzo evolutivo. Todas las ejecuciones conservaron la misma configuración base descrita en la Sección ?? y se aplicaron sobre veinte instancias independientes bajo un

horizonte de programación de 480 minutos.

Los resultados promedio obtenidos se presentan en la Tabla ??.

Tabla 6.2: Resultados promedio del AG para distintos números de generaciones (480 minutos).

Generaciones	Tareas ejecutadas	Tareas rechazadas	Makespan	Carga total
300	31.40	6.75	178.58	758.80
800	29.90	8.55	174.33	722.60
1300	28.40	8.65	171.40	696.65

Los resultados muestran que el mejor rendimiento global se obtiene con 300 generaciones, alcanzando tanto el mayor número promedio de tareas ejecutadas como la mayor carga total utilizada por los operarios. A medida que aumenta el número de generaciones, el algoritmo no presenta mejoras sustanciales; por el contrario, tiende a producir soluciones con menos tareas completadas, lo cual indica un comportamiento de sobre-ajuste evolutivo o pérdida de diversidad en las poblaciones tardías.

Este patrón sugiere que el AG converge rápidamente en el problema de programación del taller, y que extender el proceso evolutivo más allá de 300 generaciones no aporta beneficios relevantes en términos de calidad de solución. Considerando tanto el desempeño observado como la eficiencia computacional, se seleccionaron **300 generaciones** como valor óptimo para las siguientes etapas del proceso de calibración del algoritmo genético.

6.5.7. Evaluación de la Probabilidad de Mutación

El último parámetro evaluado en el proceso de calibración del algoritmo genético corresponde a la probabilidad de mutación. Este parámetro controla el grado de exploración del espacio de búsqueda mediante modificaciones aleatorias en los genes del individuo. Una mutación demasiado baja puede conducir a una convergencia prematura, limitando la capacidad del algoritmo para escapar de soluciones subóptimas; por el contrario, una mutación excesiva puede introducir demasiada variabilidad y destruir estructuras útiles adquiridas durante el proceso evolutivo.

Siguiendo prácticas habituales en la literatura, se evaluaron tres niveles representativos de intensidad de mutación: 5 %, 15 % y 25 %. Estos valores permiten analizar la sensibilidad del AG frente a distintos grados de exploración aleatoria. Las ejecuciones conservaron la configuración base descrita en

la Sección ??, utilizando el operador de cruce previamente seleccionado y un horizonte operativo de 480 minutos. Cada configuración fue evaluada sobre veinte instancias independientes.

Los resultados promedio obtenidos se presentan en la Tabla ??.

Tabla 6.3: Resultados promedio del AG para diferentes probabilidades de mutación (480 minutos).

Probabilidad de mutación	Tareas ejecutadas	Tareas rechazadas	Makespan	Carga total
5 %	34.90	10.35	178.65	832.00
15 %	30.20	6.65	168.75	712.30
25 %	32.00	8.00	181.45	786.30

Los resultados revelan diferencias marcadas entre los niveles de mutación evaluados. Una probabilidad baja (5 %) produce el mayor número promedio de tareas ejecutadas y la mayor carga total, lo cual sugiere que, en este conjunto de instancias, una exploración moderada favorece la utilización intensiva del horizonte disponible. Sin embargo, esta configuración también genera un mayor número de tareas rechazadas, lo que indica una mayor frecuencia de incumplimientos operativos.

En contraste, el nivel intermedio del 15 % obtiene la menor cantidad de tareas rechazadas y el makespan más bajo, evidenciando soluciones más estables y con menos violaciones a las restricciones del taller. Aunque ejecuta menos tareas que la configuración del 5 %, sus cronogramas son más consistentes y equilibrados.

Finalmente, una mutación más elevada (25 %) logra un desempeño intermedio en todas las métricas, reflejando un punto donde el aumento de aleatoriedad comienza a afectar la estabilidad evolutiva sin llegar a generar beneficios adicionales.

En conjunto, estos resultados muestran un compromiso entre maximizar tareas ejecutadas y mantener la factibilidad operativa. Dado que la función de aptitud prioriza la ejecución efectiva dentro de las restricciones, se seleccionó una probabilidad de mutación del **15 %** como valor óptimo para la configuración final del algoritmo genético, al ofrecer el mejor equilibrio entre exploración, estabilidad y cumplimiento operativo.

6.6. Configuración Final del AG

A partir del proceso de calibración presentado en las subsecciones anteriores —donde se evaluaron de forma independiente el operador de cruce, el número de generaciones y la probabilidad de mutación— se obtuvo una configuración final del algoritmo genético que equilibra de manera adecuada la calidad de las soluciones y el costo computacional. Esta configuración fue utilizada en todas las comparaciones realizadas posteriormente contra el método SPT en el Capítulo 7.

En síntesis, la calibración determinó que:

- El operador de cruce *single_point* ofrece el mejor desempeño en un horizonte amplio de 480 minutos, superando consistentemente a *two_points*.
- Un número de **300 generaciones** es suficiente para que el algoritmo converja hacia soluciones de alta calidad, mientras que valores mayores no aportan mejoras significativas y pueden incluso degradar el rendimiento.
- Una probabilidad de mutación del **15 %** logra el mejor equilibrio entre exploración y explotación del espacio de búsqueda, evitando tanto la convergencia prematura (como sucede con 5 %) como la pérdida de estabilidad evolutiva (observada con 25 %).

Con base en estos resultados, la configuración final del algoritmo genético utilizada en las simulaciones comparativas es la siguiente:

Tabla 6.4: Configuración final seleccionada para el algoritmo genético.

Parámetro	Valor seleccionado
Operador de cruce	<i>single_point</i>
Número de generaciones	300
Tamaño de población	100
Método de selección	<i>rank</i>
Padres por generación	10
Padres preservados	5
Probabilidad de mutación	15 %
Semilla aleatoria	42

Esta configuración proporciona un desempeño robusto en términos de tareas ejecutadas, carga total y estabilidad del proceso evolutivo. Además, se mantiene dentro de límites razonables de tiempo de ejecución, lo que facilita su aplicación práctica en un taller industrial. Los experimentos del capítulo siguiente emplean exclusivamente esta configuración, garantizando que la comparación frente al método SPT se realice bajo condiciones justas y reproducibles.

Capítulo 7

Analisis de Resultados

7.1. Métricas utilizadas

7.1.1. Tareas ejecutadas

7.1.2. Tareas rechazadas

7.1.3. Sobrecarga (carga total)

7.1.4. Balance de carga (desviación estándar)

7.1.5. Makespan

7.2. Comparación AG vs SPT

7.2.1. Descripción general de los resultados

7.2.2. Análisis de distribución

7.2.3. Tareas ejecutadas y rechazadas

7.2.4. Sobrecarga vs eficiencia del sistema

7.2.5. Balance entre operarios

7.2.6. Ventajas y desventajas de cada enfoque

7.3. Discusión de resultados

Capítulo 8

Conclusiones

8.1. Cumplimiento de objetivos

8.2. Conclusiones técnicas

8.3. Impacto industrial

Capítulo 9

Trabajo Futuro

- 9.1. Optimización del algoritmo genético
- 9.2. Sistemas híbridos
- 9.3. Integración con el ERP
- 9.4. Pruebas con datos reales históricos

Capítulo 10

Glosario

Bibliografía

- [1] N. SHIVASANKARAN y P. SENTHILKUMAR. “SCHEDULING OF MECHANICS IN AUTOMOBILE REPAIR SHOPS USING ANN”. En: *IJCSE* (2014), págs. 55-56.
- [2] César Argüelles López, Ligia Herrera Franco y Pedro Jàcome Onofre. “Estrategia de mejora al proceso productivo de Talleres Industriales de maquinados”. En: *Universo de la Tecnológica* (2018), págs. 5-6.
- [3] Kenneth C. Laudon y Jane P. Laudon. *Management Information Systems: Managing the Digital Firm*. 13.^a ed. Pearson, 2014.
- [4] Jairo R. Coronado-Hernandez et al. “Implementation of an E.R.P. Inventory Module in a Small Colombian Metalworking Company”. En: *Sprinter* (2019).
- [5] Rahul Malhotra, Narinder Singh y Yaduvir Singh. “Genetic Algorithms: Concepts, Design for Optimization of Process Controllers”. En: *CCSE* (2011), pág. 39.
- [6] Yunior César Fonseca Reyna y José Eduardo Márquez Delgado. “Comparación de un algoritmo genético y la técnica nube de partículas en la solución del Flow Shop Scheduling”. En: *Revista Cubana de Ciencias Informáticas* 6.4 (2012), págs. 17-26.
- [7] Alexander Alberto Correa Espinal, Elkin Rodríguez Velásquez y María Isabel Londoño Restrepo. “Secuenciación de operaciones para configuraciones de planta tipo Flexible Job Shop: Estado del arte”. En: *Revista Avances en Sistemas e Informática* 5.3 (2008), págs. 151-161.
- [8] Wenqiang Zhang et al. “Metaheuristics for multi-objective scheduling problems in Industry 4.0 and 5.0: a state-of-the-arts survey”. En: *Frontiers in Industrial Engineering* (2025).

- [9] José David Meisel y Liliana Katherine Prado. “Un algoritmo genético híbrido y un enfriamiento simulado para solucionar el problema de programación de pedidos Job Shop”. En: *Revista EIA* (2010), págs. 39-51.
- [10] David A. Valle. “Resolución del Job Shop Scheduling Problem mediante metaheurísticas”. Tesis de mtría. Universidad Politécnica de Madrid, 2021.
- [11] Tomal Das. “Productivity optimization techniques using industrial engineering tools: A review”. En: *ResearchGate* (2024), págs. 376-280.
- [12] Erich C. Teppan. “Types of Flexible Job Shop Scheduling: A Constraint Programming Experiment”. En: *Proceedings of the 14th International Conference on Agents and Artificial Intelligence (ICAART)*. 2022, págs. 516-523.
- [13] A. Delgado et al. “Aplicación de algoritmos genéticos para la programación de tareas en una celda de manufactura”. En: *Ingeniería e Investigación* 25.2 (2005), págs. 24-31. URL: http://www.scielo.org.co/scielo.php?script=sci_arttext&pid=S0120-56092005000200003.
- [14] Aleksandar Savić et al. “Genetic Algorithm Approach for Solving the Task Assignment Problem”. En: *Serdica Journal of Computing* (2008), págs. 101-110.
- [15] J. Wang. “Application of Genetic Algorithms in Automated Mechanical Design”. En: *Proceedings of Innovative Computing 2024, Vol. 4*. 2024.